

Generative Modeling of Structured CAD Geometry Using Transformer and VAE Frameworks

Sebastian Lochmann

Matriculation Number: 20-925-020

Thesis Supervisors:

Matthew Keeler, IDEAL lab

Dr. Daniel Wälchli, Manukai AG

In Fulfillment of the Requirements
for the Degree of

Master of Science ETH in Computational Science and Engineering



ETH zürich

Manukai

Swiss Federal Institute of Technology Zurich

Department of Mechanical and Process Engineering (D-MAVT)

IDEAL – Chair of Artificial Intelligence in Engineering Design

Manukai AG - ETH-Spinoff for AI Manufacturing

March 2026

Spring Semester 2026

Abstract

This semester project investigates parameter-efficient fine-tuning of generative models for Computer-Aided Design (CAD) geometry generation. Specifically, we apply Low-Rank Adaptation (LoRA)—a parameter-efficient method introducing trainable rank decompositions—to the AutoBrep transformer-based model for domain-specific adaptation. We develop a complete pipeline for preprocessing CAD data, fine-tuning with LoRA adapters, and generating novel Boundary Representations. We evaluate training convergence, quantitative metrics, and scalability of LoRA adapters.

Acknowledgments

I would like to extend my gratitude to my thesis supervisors, Matthew Keeler at the IDEAL lab and Dr. Daniel Wälchli at Manukai AG, for their invaluable guidance, insightful feedback, and continuous support throughout this research. Their expertise in generative modeling and engineering design has significantly shaped the direction and quality of this work.

I thank the IDEAL lab team and Manukai AG for providing access to computational resources and data, enabling this research. Special thanks to the research communities working on generative design and parameter-efficient fine-tuning for their foundational contributions to this field.

Finally, I am grateful to my colleagues and friends for their encouragement and constructive discussions during the development of this thesis. Their feedback and support were instrumental in bringing this work to completion.

Contents

Abbreviations	v
1 Introduction	1
1.1 Context and Motivation	1
1.2 Problem Statement	1
1.3 Thesis Objective	1
1.4 Contributions	2
1.5 Thesis Structure	2
2 Background and Related Work	3
2.1 Boundary-Representation Fundamentals	3
2.2 Generative Models for CAD	3
2.2.1 BrepGen: Diffusion-Based Approach	3
2.2.2 AutoBrep: Autoregressive Approach	4
2.2.3 NeuroNurbs: NURBS Parameter Encoding	5
2.2.4 HoLa: Holistic Latent Representation	5
2.3 Low-Rank Adaptation (LoRA)	6
3 Methodology	8
3.1 AutoBrep Transformer Architecture	8
3.2 LoRA Fine-tuning Configuration	8
3.3 Data Preparation Pipeline	9
3.4 Training Setup	10
3.5 Inference and B-Rep Generation	11
4 Experimental Results	12
4.1 Baseline Inference	12
4.1.1 Comparison with BrepGen	12
4.2 Training Convergence	13
4.3 Empirical Findings on LoRA Fine-Tuning Dynamics	13
4.4 Quantitative Evaluation	14
4.5 Qualitative Results	15
4.6 Ablation Studies	17
5 Discussion of Research Questions	20
5.1 RQ1: Capabilities and Limitations of AutoBrep	20
5.2 RQ2: Efficient Domain Adaptation with Limited Data	20
5.3 RQ3: Sequential and Sparse (Auto)completion	20
5.4 Limitations and Future Research	20
6 Conclusion	21

References 22

A Appendix 23

A.1 AI Use Declaration 23

Abbreviations

B-Rep	Boundary Representation — explicit representation of 3D solid surfaces via faces, edges, and vertices
LoRA	Low-Rank Adaptation — parameter-efficient fine-tuning technique using rank- r decompositions
CAD	Computer-Aided Design — software tools for designing and modeling engineering geometry
STEP	Standard for the Exchange of Product Model Data — ISO 10303 file format for CAD models
UV-Grid	2D parametric coordinate system on a B-Rep face surface
AutoBrep	State-of-the-art transformer-based generative model for B-Rep synthesis
NURBS	Non-Uniform Rational B-Splines — parametric surface/curve representation standard in CAD
VAE	Variational Autoencoder — generative model learning compressed latent representations
GNN	Graph Neural Network — neural architecture operating on graph-structured data
W&B	Weights & Biases — ML experiment tracking and visualization platform

1 Introduction

1.1 Context and Motivation

This thesis is conducted at the IDEAL lab in collaboration with Manukai AG, an ETH spinoff developing machine learning models for CAM manufacturing. Manukai faces a critical challenge: they need domain-specific synthetic CAD training data to fine-tune their models, but cannot mix customers' proprietary CAD datasets in their model between companies due to trade secret concerns. Generating high-quality, domain-specific data is therefore essential.

Recently, the state-of-the-art AutoBrep model Xu et al. (2025) demonstrated superior capabilities in generating Boundary Representations (B-Reps) with higher quality and complexity compared to prior approaches like BrepGen Xu et al. (2024). However, directly applying pre-trained generative models to new specialized domains requires either full retraining (computationally expensive and data-intensive) or efficient adaptation techniques. This motivates investigating parameter-efficient fine-tuning using Low-Rank Adaptation (LoRA) to enable rapid domain adaptation with limited data and compute.

1.2 Problem Statement

Machine learning models for CAD geometry generation have been developed, including the state-of-the-art AutoBrep model, which uses transformer-based architectures to generate Boundary Representations (B-Reps) autoregressively. However, applying such models to new, specialized design domains presents significant challenges:

Key challenges:

- **Domain Adaptation:** Pre-trained models reflect patterns from their training data; specialized design domains (e.g., specific mechanical parts, architectural elements) may have unique characteristics not well-represented in general training corpora
- **Limited Data:** High quality CAD data which is B-Rep compatible is scarce and expensive. It exists in large quantities in industrial companies, but often as parts of trade secrets, and is created in work-intensive manual work by experts.

1.3 Thesis Objective

This work addresses these challenges by investigating *parameter-efficient fine-tuning* of generative CAD models using Low-Rank Adaptation (LoRA). The core research question is:

1. What are the **capabilities and limitations** for the AutoBrep CAD GPT model published by Xu et al. (2025)?
2. Can the model be efficiently **adapted to specialized design domains** with limited data?

3. Does the model allow for sequential or sparse **(auto)completion** of solids?

To answer this, we implement a complete pipeline for LoRA-based fine-tuning of the AutoBrep model, deploy it on Euler GPU's, explore hyperparameters, tracking with W&B. Additionally, we try out (auto)completion. Eventually we evaluate our results.

1.4 Contributions

This thesis makes the following contributions to the field of generative design and machine learning for CAD:

1. **Practical Framework:** An end-to-end implementation of LoRA fine-tuning for AutoBrep, including data preprocessing, training, and inference pipelines
2. **Viability Analysis:** Empirical demonstration whether domain-specific performance can be achieved with only $\approx 1\%$ additional trainable parameters and order of 500 samples, enabling fine-tuning with very limited data and without changing the base model's parameters
3. **Reproducibility:** Detailed documentation of hyperparameters, code, training curves and experimental procedures enabling future research and practical adoption. All code is available at https://github.com/slochmann/semester_project_autobrep.

1.5 Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 2:** Background and related work on B-Reps, generative models for CAD, LoRA, and related domain adaptation techniques
- **Chapter 3:** Detailed methodology covering AutoBrep architecture, LoRA integration, data preprocessing, and training procedures
- **Chapter 4:** Experimental results including training convergence, quantitative evaluation, and qualitative analysis of generated models
- **Chapter 5:** Discussion of findings, limitations, and directions for future research
- **Chapter 6:** Summary of contributions and practical implications

2 Background and Related Work

2.1 Boundary-Representation Fundamentals

The Boundary Representation (B-Rep) is a fundamental data structure in Computer-Aided Design (CAD) systems. B-Reps describe 3D solid objects by explicitly representing their boundary surfaces using parametric equations (e.g. Non-Uniform Rational Basis Splines), enabling precise surface definition. The key components include faces, edges and vertices. They maintain topological consistency by preserving relationships between geometric elements, ensuring valid and watertight solid structures. As an industry standard, B-Reps are widely supported by professional CAD software including Autodesk, Solidworks, and FreeCAD.

2.2 Generative Models for CAD

Two complementary approaches have emerged for direct B-Rep synthesis: diffusion-based and autoregressive methods. This section compares the two approaches based on two foundational papers.

2.2.1 BrepGen: Diffusion-Based Approach

Xu et al. (2024) introduces BrepGen, a diffusion-based model for direct B-Rep generation. The key innovation is a *structured latent geometry* representation that encodes B-Reps as hierarchical trees with three levels: faces, edges, and vertices.

Latent Space Encoding: B-Reps are converted into point cloud representations by sampling 32×32 points uniformly over each surface and 32 points over each edge. This hierarchical tree structure explicitly encodes topology—connections between faces and edges—enabling diffusion models to learn both geometry and topology jointly.

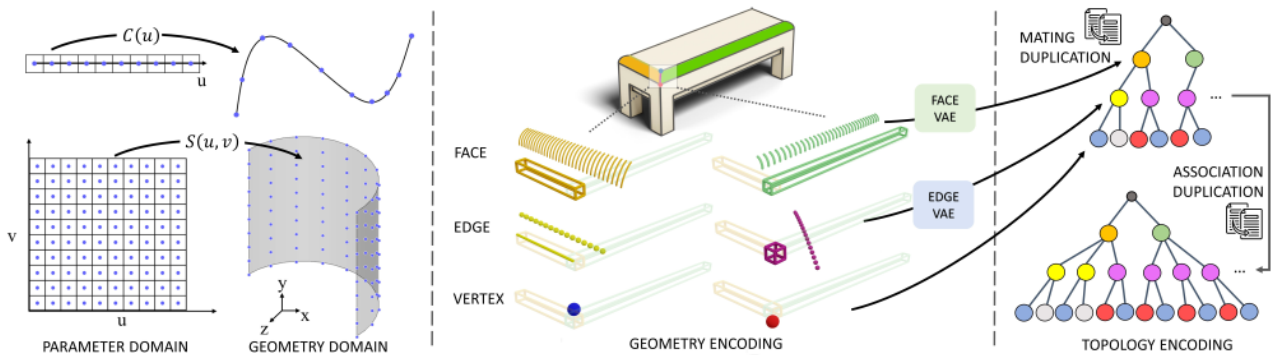


Figure 1: BrepGen encoding: B-Reps are converted into point clouds with 32×32 samples per face and 32 per edge, organized in a hierarchical tree encoding topology Xu et al. (2024)

Generation Pipeline: BrepGen employs a multi-stage diffusion approach with six modules (2 VAEs + 4 denoisers). Noise for faces and bounding boxes is denoised sequentially. Conditioned on these refined faces, edge noise is iteratively refined.

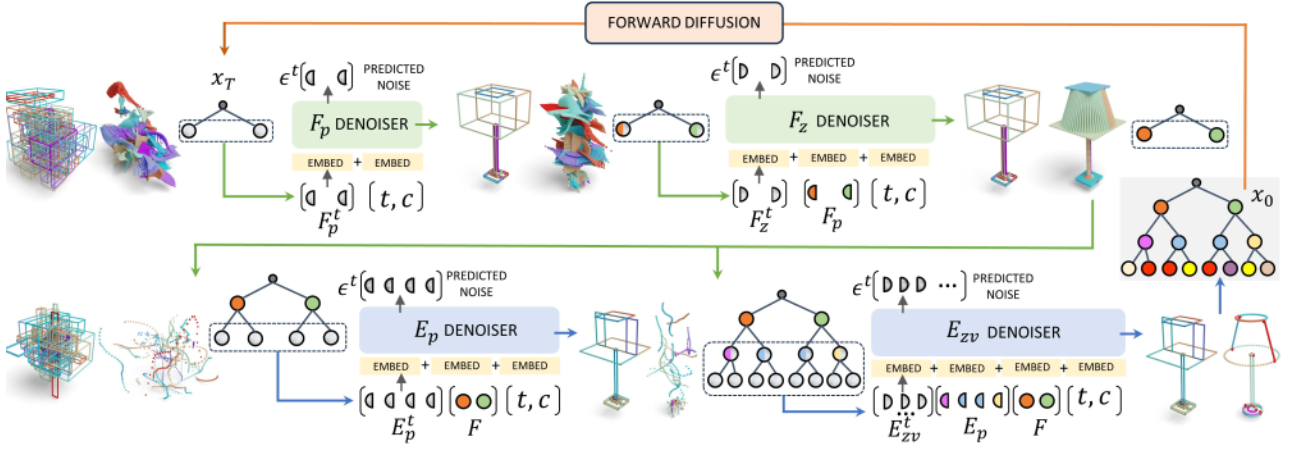


Figure 2: BrepGen generation: Multi-stage diffusion with separate denoisers for faces (bounding boxes) and edges, conditioned on intermediate face predictions Xu et al. (2024)

Post-Processing (Heuristic + Optimization): The generated point clouds require extensive post-processing:

1. Deduplication of vertices using geometric thresholds (0.08 bounding box tolerance, 0.2 point distance)
2. Recovery of topological associations (mating of shared geometry across faces)
3. Geometric refinement: averaging vertex positions, aligning/flipping edges, fitting surfaces using OpenCascade
4. Final sewing into valid solids in .step format

Performance: BrepGen achieves moderate validity rates on simpler geometries but shows reduced performance on high-complexity solids (50+ faces). Its multi-stage cascading pipeline is slower than autoregressive approaches.

2.2.2 AutoBrep: Autoregressive Approach

Xu et al. (2025) presents AutoBrep, an autoregressive Transformer that unifies B-Rep geometry and topology into a single stream of discrete tokens. Unlike multi-stage diffusion, AutoBrep generates B-Reps end-to-end via next-token prediction, treating CAD generation as a language modeling task.

Latent Space Encoding: AutoBrep uses a VAE to encode the point cloud representation (identical to BrepGen’s initial encoding) into discrete tokens using finite scalar quantization with a 1000-word vocabulary. These tokens are then sequenced based on a breadth-first search [3](#) ordering of the B-Reps topology, preserving structural relationships.

Key advantages: (1) Single unified architecture vs. multi-stage pipeline, (2) 2–5× faster inference via next-token prediction, (3) Native support for B-Rep completion through next token prediction, (4) Scalability to 100+ face solids.

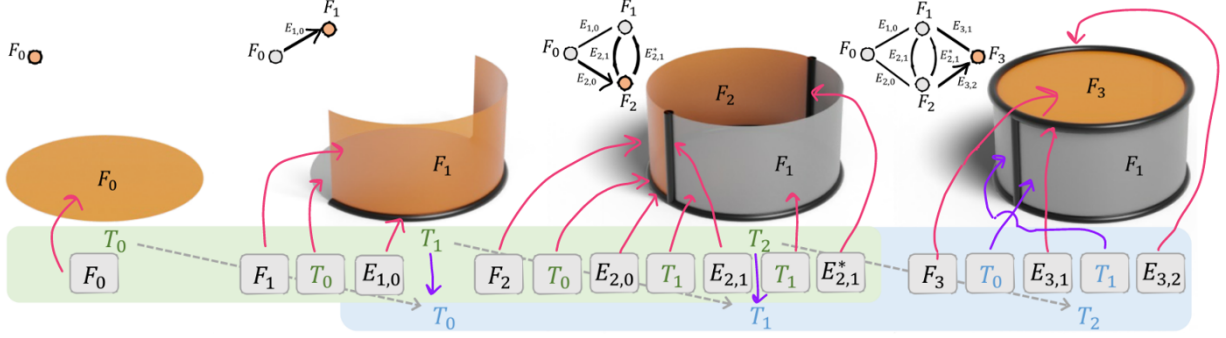


Figure 3: Breadth first search and corresponding token sequence. Pink arrows point from tokens to corresponding faces and edges. Violet arrows point to corresponding topologies Xu et al. (2025)

2.2.3 NeuroNurbs: NURBS Parameter Encoding

Fan et al. (2024) proposes NeuroNURBS, which learns surface representations by directly encoding NURBS parameters (control points, knot vectors, weights) rather than sampling UV-grids. This native representation reduces memory consumption by 79.9% and GPU training requirements by 86.7% compared to UV-grid methods. When integrated into BrepGen (yielding NURBS-Gen), it produces smoother surfaces without undulation artifacts and improves FID from 30.04 to 27.24. While offering computational efficiency gains, NeuroNURBS was not pursued in this work due to focus on autoregressive methods. However, it represents a promising direction for memory-efficient B-Rep generation.

2.2.4 HoLa: Holistic Latent Representation

Liu et al. (2025) introduces HoLa, a diffusion-based approach that unifies B-Rep geometry and topology in a single latent space. The key innovation is observing that all curves in a B-Rep arise from surface intersections, enabling HoLa to infer curve and vertex primitives from surface pairs via a neural intersection module. This eliminates the need for separate latent spaces and decoders for different primitive types, as used in methods like BrepGen.

HoLa employs a VAE encoding followed by a latent diffusion model for conditional generation from point clouds, images, sketches, and text. The holistic representation achieves 82% validity on complex solids compared to $\approx 50\%$ for multi-stage diffusion approaches, and supports diverse conditioning modalities with a single trained model.

For domain-specific fine-tuning, HoLa’s structured geometry-topology encoding may offer advantages over purely autoregressive approaches: the explicit intersection module could provide stronger inductive biases for preserving topological validity during adaptation, potentially complementing LoRA-based approaches for future work.

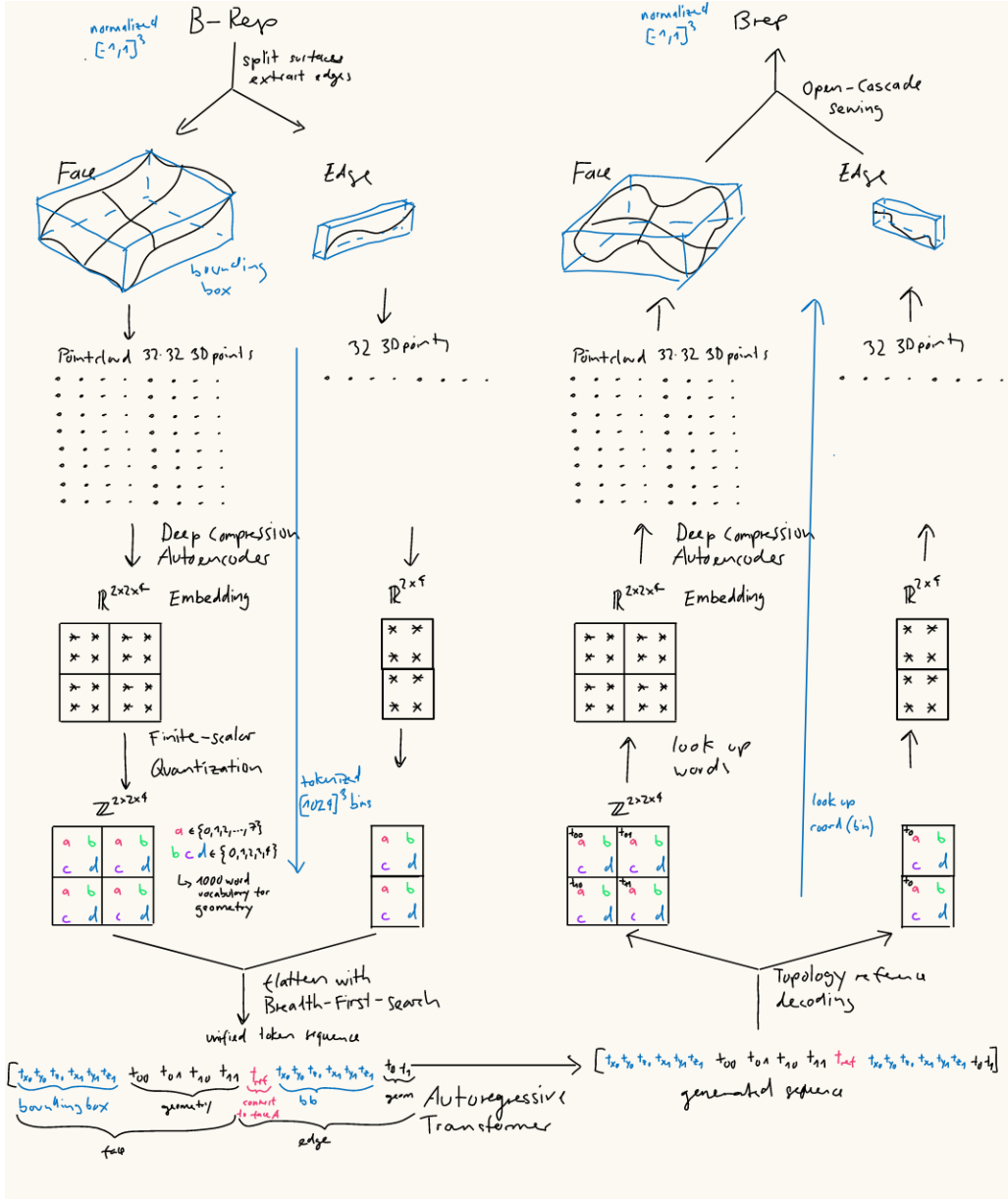


Figure 4: AutoBrep architecture: Point clouds are VAE-encoded into discrete tokens, sequenced via BFS, then passed to a GPT-like causal-masked transformer for next-token prediction. Blue boxes, arrows and sequence part corresponds to bounding box.

2.3 Low-Rank Adaptation (LoRA)

LoRA (Low-Rank Adaptation) is a parameter-efficient fine-tuning technique introduced by Hu et al. (2021). Rather than fine-tuning all model parameters, LoRA introduces small trainable low-rank decompositions in the projection matrices of attention layers within a Transformer. This approach dramatically reduces memory requirements and training time while maintaining competitive or superior performance relative to full fine-tuning.

The fundamental idea is to decompose weight updates as a product of two low-rank matrices which are then added to the frozen pre-trained weights Fig 5. This approach is very compute and memory efficient by only adding adapters of around 1% model size to the base model while keeping base model weights constant.

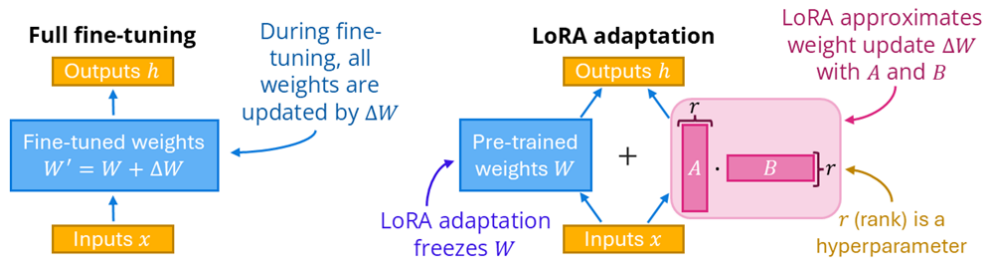


Figure 5: Full fine-tuning vs LoRA adaptation Hu et al. (2021).

3 Methodology

3.1 AutoBrep Transformer Architecture

AutoBrep is a decoder-only transformer model with approximately 1 billion parameters, designed for autoregressive generation of Boundary Representations (B-Reps). The architecture consists of:

- **35-layer Transformer decoder** with embedding dimension $d = 768$
- **Multi-head attention**: Processes query (`to_q`), key, and value (`to_v`) projection layers
- **Position-wise feedforward networks (MLP)**: Projections $768 \rightarrow 16384 \rightarrow 768$
- **Tokenization layers**: Meta tokens (complexity class), CAD tokens (face/edge/topology sequences), and geometry tokens

The model is trained on approximately 1 million B-Reps and achieves reported validity rates up to 50% for complex geometries (up to 100 faces). It operates autoregressively, predicting the next token conditioned on all preceding tokens in the sequence.

3.2 LoRA Fine-tuning Configuration

Following the LoRA approach described in Chapter 2, we apply low-rank adapters to the AutoBrep transformer architecture. Rather than full fine-tuning, only adapter parameters are trainable while the 1B-parameter base model remains frozen.

Adapter Placement and Dimensions:

$$\mathbf{z}_q = \mathbf{W}_q \cdot \mathbf{x} + \frac{\alpha}{r} \cdot \mathbf{x} \cdot \mathbf{B} \cdot \mathbf{A}$$

where $\mathbf{A} \in \mathbb{R}^{r \times 768}$, $\mathbf{B} \in \mathbb{R}^{768 \times r}$.

AutoBrep is a 35-layer decoder-only Transformer with embedding dimension $d = 768$. We explored two adapter placement strategies:

1. **Attention-Only** (Primary): Adapters inserted into query (`to_q`) and value (`to_v`) projection layers:
2. **Attention + Feedforward** (Ablation): Additional adapters on the feedforward projection layers $\text{ff} : \mathbb{R}^{768} \rightarrow \mathbb{R}^{16384}$ to capture more model capacity.

Each adapter pair at rank r contributes trainable parameters. We explored two configurations:

Attention-Only Configuration: Across 35 layers with 2 adapters per layer ($35 \times 2 = 70$ total), with $d = 768$.

Attention + Feedforward Configuration: Additionally inserting adapters into the feedforward projections ($768 \rightarrow 16384 \rightarrow 768$) adds substantial capacity.

Across 35 layers: $\text{Extra} = 35 \times 17,152 \times r = 600,320 \times r$ parameters.

Table 1: LoRA Parameter Efficiency: Attention-Only vs. Attention + Feedforward

Config	Rank (r)	Params (M)	% Base
Attention	4	0.43	0.043%
	8	0.86	0.086%
	16	1.72	0.172%
	32	3.44	0.344%
Attention + FF	16	12.12	1.212%
	32	24.24	2.424%

Even with feedforward adapters at $r = 32$, total parameters remain $< 2.5\%$, but at significant cost in memory and training time. Attention-only adapters achieved comparable results with minimal overhead.

3.3 Data Preparation Pipeline

A custom data processing pipeline converts raw STEP files (Fig. 6) into a structured parquet dataset compatible with AutoBrep tokenization:

Processing steps:

1. **STEP Loading and Feature Extraction:** Parse CAD solids using Open Cascade, extracting faces, edges, and topology information
2. **UV-Grid Sampling:** Sample UV-parameter spaces to create dense point clouds representing geometric surfaces
3. **Geometry Normalization:** Normalize coordinates to the $[-1, 1]^3$ range bounding box (Fig. 7)
4. **Serialization:** Convert numpy arrays to bytes for compact storage in the dataset
5. **Parquet Export:** Write preprocessed geometries with corresponding bounding boxes, complexity metadata (face count), and split into train/validation (0.9/0.1) splits

The resulting parquet dataset contains serialized geometry tensors (faces, edges) with corresponding bounding box annotations and topology metadata.

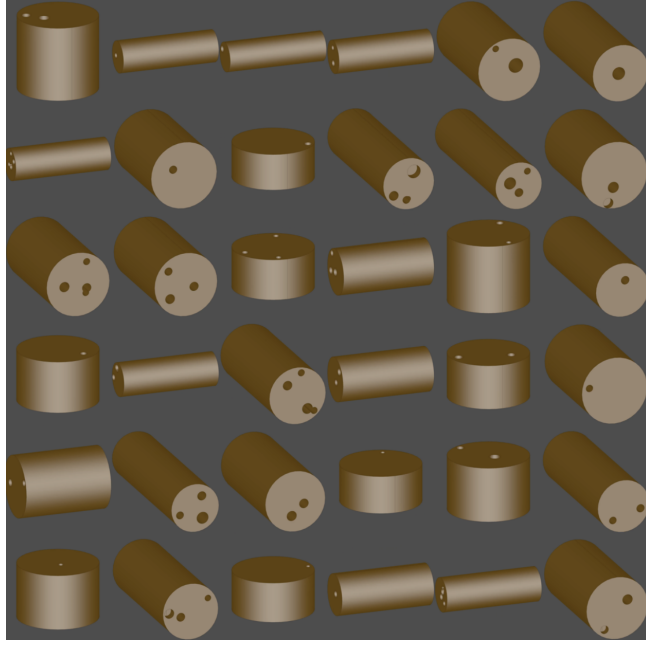


Figure 6: Raw .step file renderings of the generic cylinders with holes dataset provided by Manukai

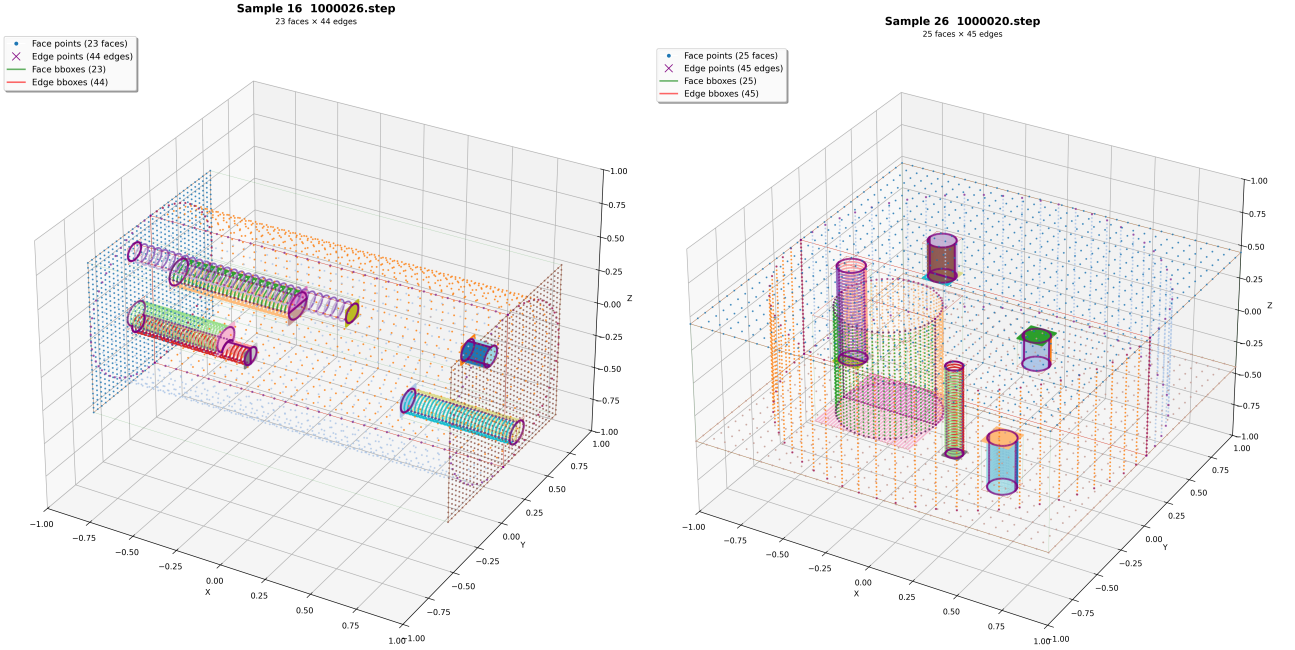


Figure 7: Two examples of preprocessed point cloud solids of the generic cylinders with holes

3.4 Training Setup

Fine-tuning is performed using the AdamW optimizer on the frozen base model with LoRA adapters as the only trainable parameters. Hyperparameter selection in table 2 follows established LoRA fine-tuning practices Hu et al. (2021) and UnSloth (2024).

GPU Memory Optimization: Training on a single RTX 4090 GPU (24 GB) was enabled through three techniques: (1) casting the model to bfloat16 precision to reduce memory overhead, (2) using PyTorch’s expandable memory segments

Table 2: Training Hyperparameters following Hu et al. (2021)

Parameter	Value	Notes
Batch Size	4	Same as in Autobrep Xu et al. (2025)
Number of Epochs	4	Typical
Learning Rate	1e-4 to 5e-5	Lower rates recommended for small datasets to prevent overfitting
LoRA Rank	8–32	Balance between model capacity and parameter efficiency
LoRA Alpha	$\approx 4 \times r$	Effective scaling = α/r ; adjust for sensitivity
Optimizer	AdamW	Standard choice with weight decay for regularization

(`PYTORCH_ALLOC_CONF=expandable_segments:True`) to avoid fragmentation, and (3) periodic GPU cache cleanup (`torch.cuda.empty_cache()`) every 10 batches. These techniques allowed efficient training without requiring gradient accumulation.

The train and validation loss function is computed using cross-entropy loss from the AutoBrep module, which measures the divergence between predicted and ground-truth token distributions.

Training is monitored via Weights & Biases (W&B) for real-time loss tracking, learning rate schedules, and system metrics.

3.5 Inference and B-Rep Generation

During inference, the fine-tuned model generates B-Rep sequences autoregressively, conditioned on a complexity token. Generation is controlled via temperature sampling; top- p (nucleus) sampling is kept constant at 0.95 to ensure diverse yet reasonable outputs. During training unconditional samples are created after each epoch. Token sequences are decoded into geometric primitives (surfaces, curves) and topological relationships via the FSQ-VAE decoders. The reconstructed B-Rep geometry is then validated and assembled into watertight solids (STEP files) using the AutoBrepBuilder with the following parameters:

Table 3: Inference and B-Rep Reconstruction Parameters

Parameter	Value	Interpretation
<i>Inference Control</i>		
Complexity Token	14–17	14: <25 faces, 15: <50 faces, 16: >50 faces, 17: random
Temperature	0.3–0.8	Controls sampling diversity; 0.5–0.7 found optimal for fidelity+diversity
<i>Reconstruction Tolerances</i>		
Sewing Tolerance	2×10^{-3}	Max distance for edge-face sewing (watertightness)
Vertex Threshold	2×10^{-3}	Max separation for vertex merging
Z-Threshold	0	Z-coordinate deviation check enforced to zero (strict)

A B-Rep is considered valid if it passes the solid modeling kernel’s validation checks: all edges are shared by exactly 2 faces (no dangling edges) and the topology forms a closed manifold (watertight) within tolerances.

4 Experimental Results

4.1 Baseline Inference

In Fig. 8 we can see that the unmodified AutoBrep model indeed generates impressively complex and watertight B-Reps. Each of these STEP samples was rendered from various angles and concatenated in a GIF showing them from all directions for inspection.

The generated solid structures appear coherent with few exceptions of small gaps between faces or non-sensical structure in $< 10\%$ of samples.

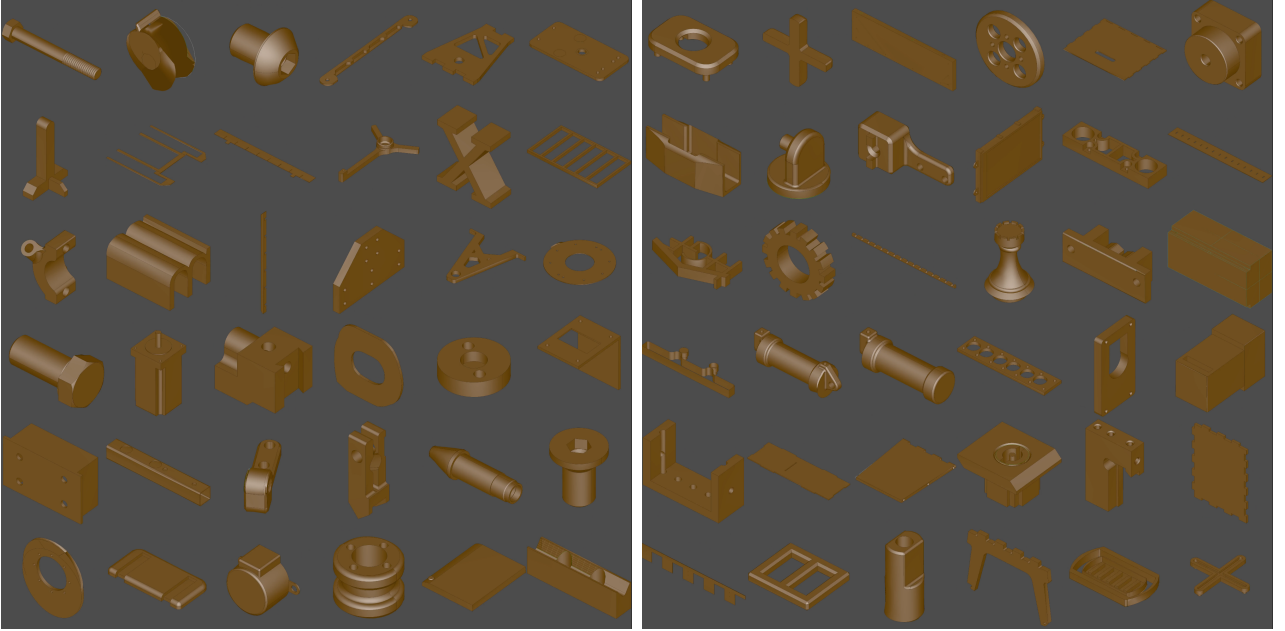


Figure 8: Examples of unconditional samples generated with the untuned base model checkpoint. Left with medium complexity (25 to 50 faces per shape) and right with high complexity (50+ faces per shape).

4.1.1 Comparison with BrepGen

To contextualize AutoBrep’s performance, we compare it with BrepGen, the predecessor diffusion-based approach discussed in Chapter 2. Figure 9 shows representative samples generated with BrepGen.

AutoBrep offers several key advantages over BrepGen: higher geometric complexity with diverse topological structures versus BrepGen’s simple geometries (cylinders, cuboids with chamfers and holes), superior feature diversity across complexity ranges, 2–5 $\times$ faster generation speed through single-stage autoregression, and better scalability supporting up to 100+ face solids.

Base Model Constraints: During baseline inference evaluation, we observed that the unconditionally-trained base model does not support conditional generation tasks such as autocomplete or sequence completion. The provided checkpoint appears to have been trained

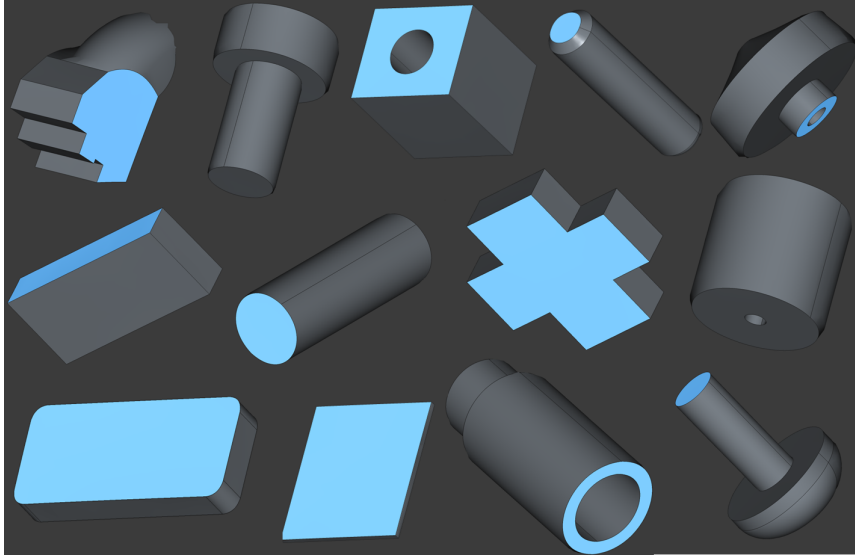


Figure 9: Representative samples generated with BrepGen (diffusion-based approach) from unconditional generation. While geometrically valid, BrepGen produces predominantly simple geometries with low complexity.

without geometry tokens (which are commented out in the repository code), limiting its ability to complete partial B-Rep sequences.

4.2 Training Convergence

The LoRA fine-tuning process was conducted on custom CAD model datasets, following the training setup and hyperparameters specified in Chapter 3.

Observed Training Characteristics:

- Best results were around a loss of 1.0 within 3–7 epochs for 600 samples
- Validation loss tracked closely with training loss (typically within 10–20% separation)
- LoRA rank had minimal influence on training convergence dynamics

Training and validation loss curves are shown in Fig. 10.

4.3 Empirical Findings on LoRA Fine-Tuning Dynamics

LoRA Behavior as Distribution Sharpener: Through systematic experimentation, LoRA was observed to function as a *distribution sharpener* rather than a distribution shifter. When applied to attention layers alone, LoRA concentrates probability mass on modes already present in the base model’s feature space, but cannot learn genuinely novel geometric features outside this space. This fundamentally constrains what fine-tuning can achieve: the base model must already contain (in distributed form) the geometric features present in the fine-tuning data.

Effect of Adapter Placement: Experiments comparing attention-only adapters (as specified in Chapter 3, Table 1) with attention + feed-forward adapters revealed minimal dif-

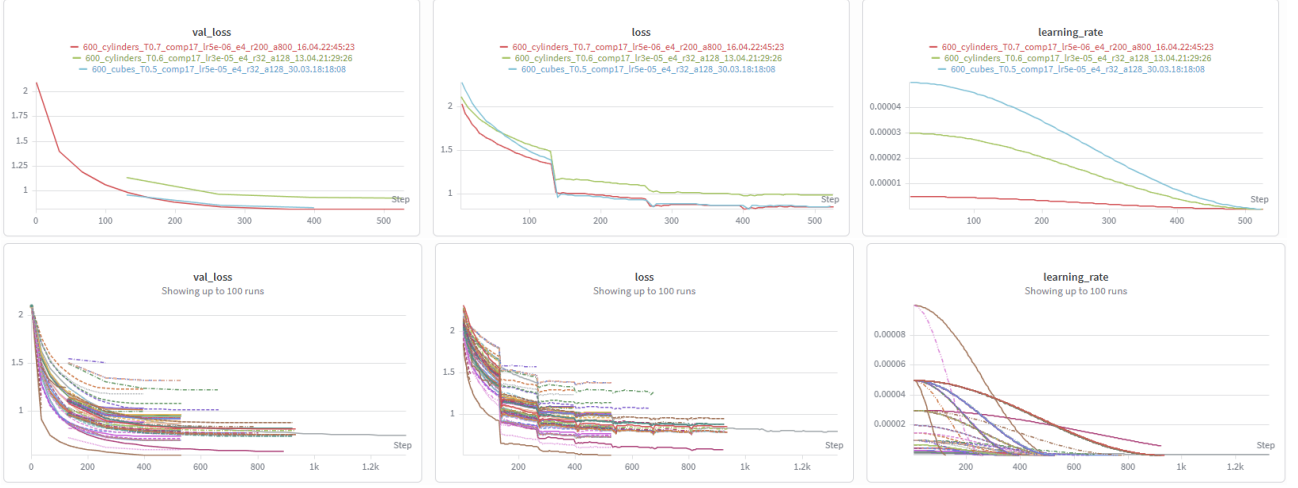


Figure 10: Training dynamics for representative and aggregated runs. Top: three selected configurations showing convergence toward loss ≈ 1.0 with validation loss tracking within 10–20% separation. Bottom: overlaid trajectories from 100 experimental runs demonstrating robust convergence patterns across hyperparameter variations.

ferences in output quality or decodability. Figure 16 illustrates the marginal improvement from adding feed-forward adapters. This finding supports the hypothesis that most adaptation occurs through attention weight redistribution, with feed-forward layers providing limited additional benefit.

4.4 Quantitative Evaluation

Decodability Rate: A critical metric for generative B-Rep models is the fraction of generated sequences that successfully decode into valid solids. Across all fine-tuning runs, the fine-tuned LoRA models achieved approximately 80% decodability rate when tuning hyperparameters appropriately. This rate remained high unless the model exhibited signs of overfitting, in which case generated sequences became degenerate and decoding failed completely.

Temperature Sensitivity: Inference temperature was varied across the range $T \in [0.3, 0.8]$ to assess its impact on output diversity and fidelity to training data. Key findings:

- **Low temperature** ($T \approx 0.3$): Model outputs concentrated on the most probable modes, with minimal diversity. Strong LoRA effect but only most basic cylinders / cuboids without any features like holes.
- **Optimal range** ($T \in [0.5, 0.7]$): Best balance between resemblance to training data and geometric diversity. Results showed prominent training features (e.g., holes, cylinders, cuboids) but not exact reproductions of training shapes.
- **High temperature** ($T > 0.8$): Increased diversity but reduced fidelity to training distribution. Model samples from broader set of modes beyond finetuning data.

Temperature had minimal impact on decoding success rate; failures were primarily driven by model overfitting rather than inference temperature.

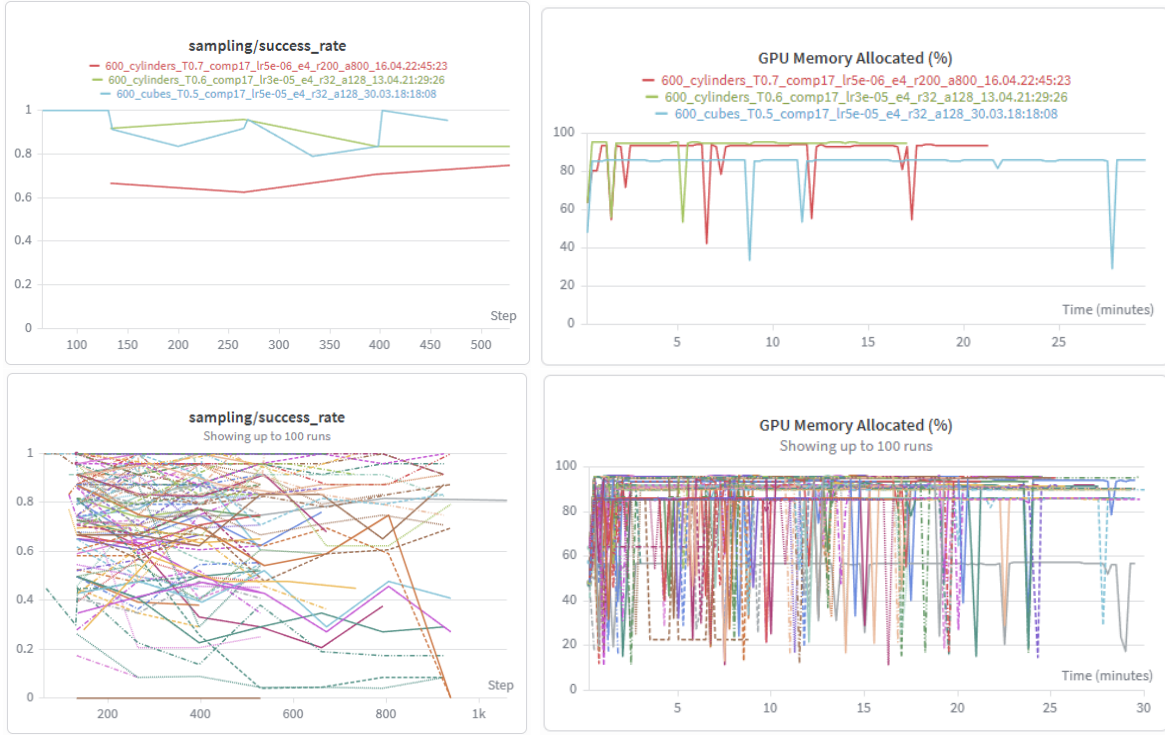


Figure 11: Decodability rate and GPU memory across configurations. Top: The three selected runs’s decodability and GPU usage (100% = 24GB). Bottom: Same for 100 experiments.

Further evaluation against baseline (untuned AutoBrep) is presented in Section 4.6.

4.5 Qualitative Results

Generated B-Reps are visualized and evaluated for geometric plausibility, feature fidelity, and diversity relative to training data.

Cylinders with Holes Dataset: Fine-tuning was performed on 600 cylindrical solids with random length and diameter and random holes. The fine-tuned model produced results featuring:

- Cylindrical geometry for approximately half of the generated samples
- Variations in hole placement and size
- Decodable, watertight solids in approximately 90% of samples

Examples are shown in Fig. 12.

Cuboids with Holes Dataset: Fine-tuning was also performed on 600 cuboid solids with random holes, producing analogous results:

- Cuboid or near-cuboid primary geometry
- Through-holes with varied placement patterns
- High decodability (approximately 90%)

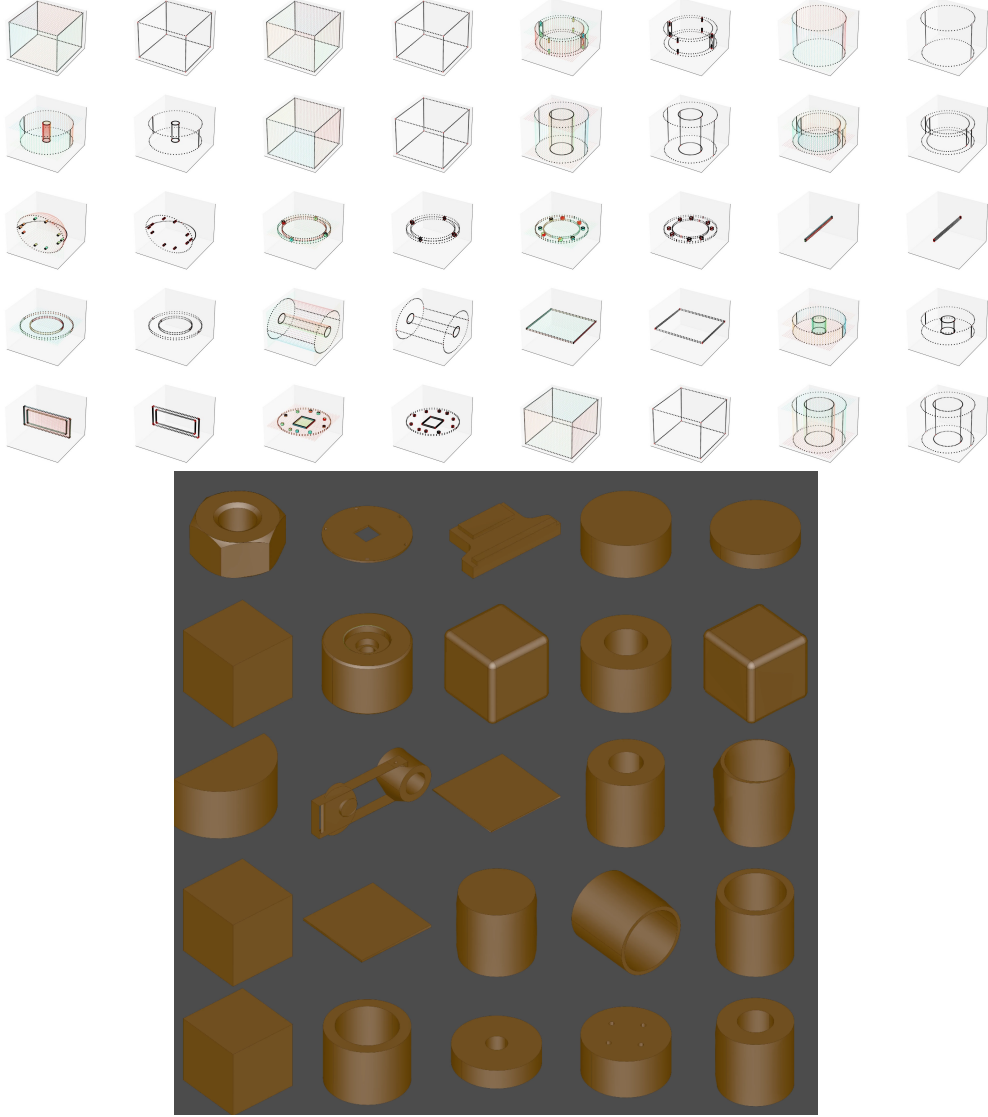


Figure 12: Qualitative Results: Cylinders with Holes. Top: Point clouds of faces (coloured) and corresponding edges (black). Bottom: Rendered STEP files. Samples exhibit characteristic training features but are not exact reproductions. LR: 3×10^{-5} , epochs: 4, rank: 32 (alpha 128), adapters: Q and V, T: 0.6, complexity: 17.

Examples are shown in Fig. 13.

Base Model vs. LoRA Comparison: To isolate the effect of fine-tuning, we compare unconditional samples from the base model at temperature $T = 0.6$ with fine-tuned LoRA samples at the same temperature, using the same random seed for reproducibility. Figure 14 shows this comparison.

Sequence Completion via Next-Token Prediction: While the fine-tuned model demonstrates some learning of training-domain geometries, it fails at sequence completion tasks. When provided with partial B-Rep sequences (e.g., only face geometry tokens representing 20% of the full sequence), the model’s next-token predictions generate degenerate edge and face geometry that does not reconstruct valid solids. Figure 15 shows representative examples of these failed completions.

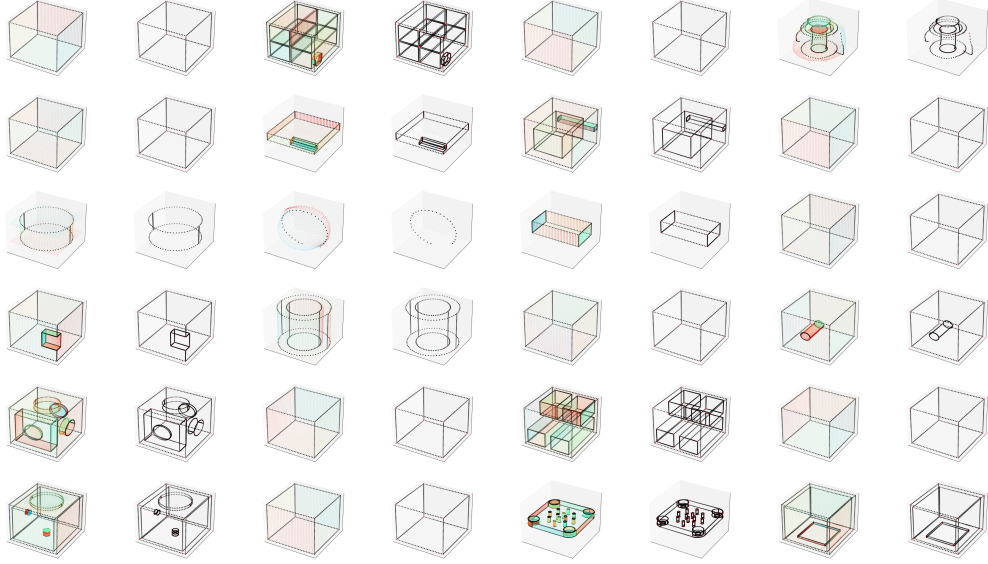


Figure 13: Qualitative Results: Cuboids with Holes. LR: 5×10^{-5} , epochs: 4, rank: 32 (alpha 128), adapters: Q and V, T: 0.5, complexity: 17.

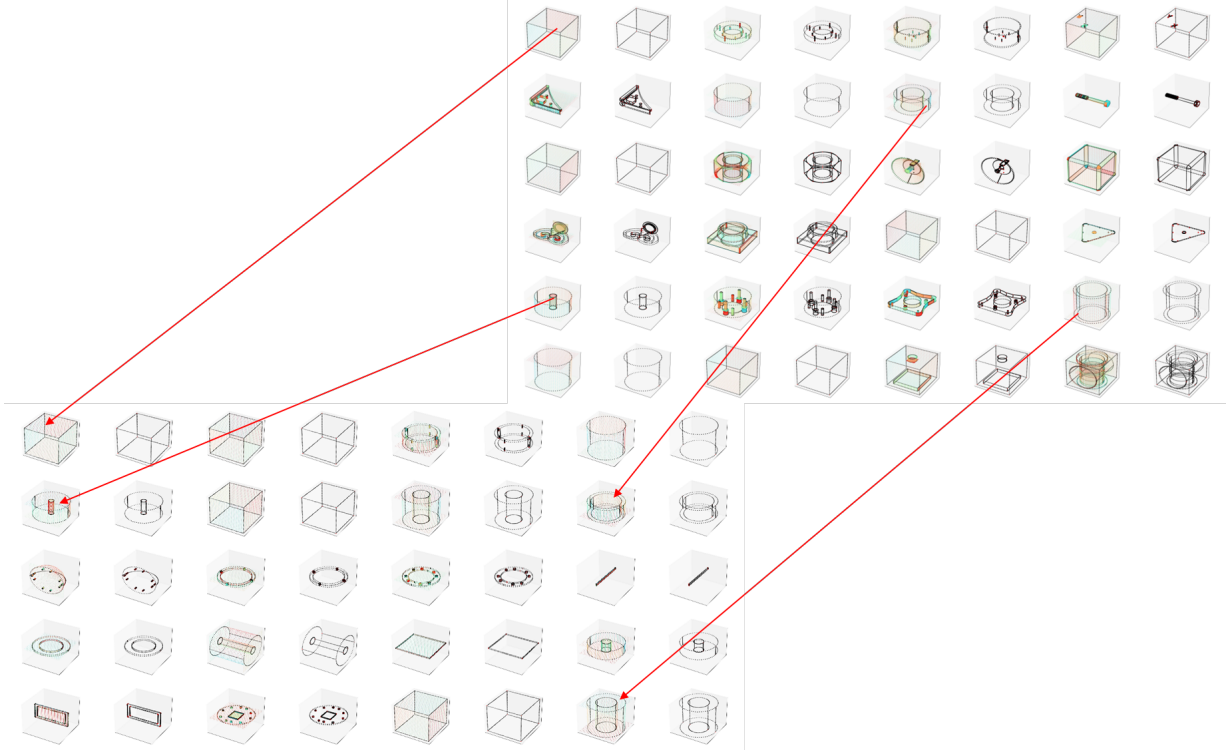


Figure 14: Base Model vs. LoRA Fine-Tuning at $T = 0.6$. Top right: unconditional samples from base AutoBrep model. Bottom left: samples from LoRA fine-tuned on cylinders-with-holes. Arrows indicate corresponding geometries appearing in both outputs, demonstrating that LoRA acts as a distribution sharpener.

4.6 Ablation Studies

To understand the contribution of different training and inference components, systematic ablation experiments were conducted.

LoRA Rank Impact: Varying LoRA rank $r \in \{16, 32\}$ showed minimal influence on

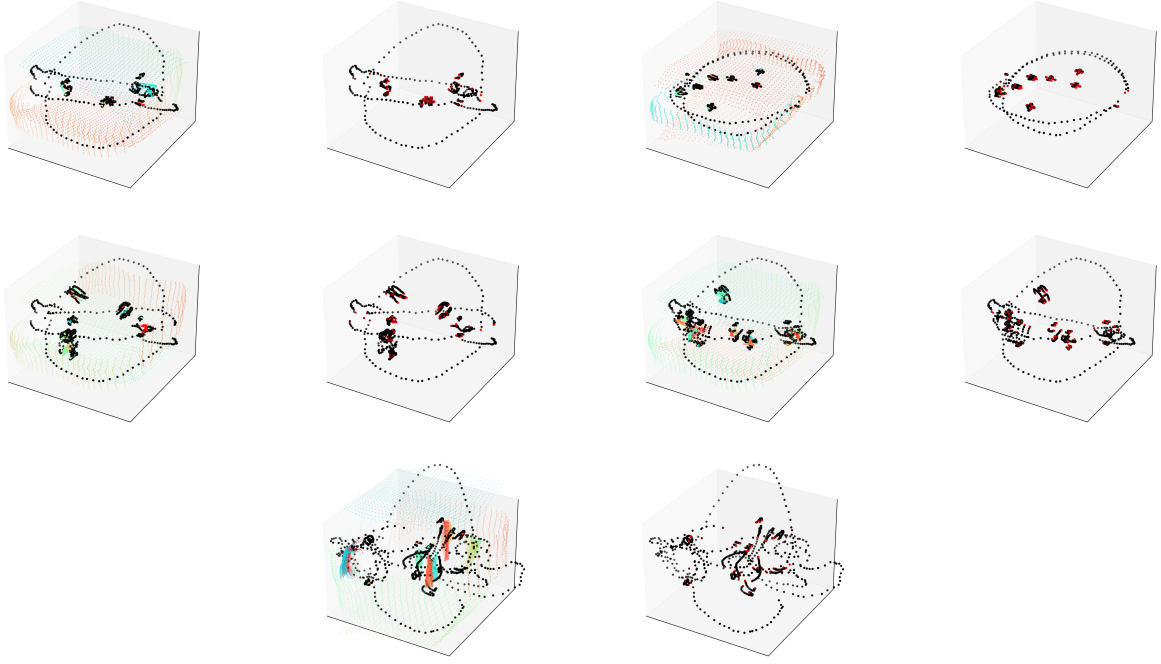


Figure 15: Failed sequence completion examples. Each row shows input face-geometry (left) and model-predicted edge-geometry (right) for five samples. Despite successful feature learning on complete sequences, the fine-tuned model fails to generate valid B-Rep completions when given partial token sequences (20% of levels, see 3).

final output quality or convergence behavior. Both configurations achieved similar decodability rates (approximately 90%) and produced outputs with comparable feature amplification. This suggests that the primary bottleneck is the base model’s learned feature space rather than adapter capacity.

Exploration of significantly higher ranks ($r = 200$) combined with feed-forward adapters demonstrated saturation effects. Figure 16 illustrates cylinders-with-holes generation at $r = 200$ with feed-forward adapters (learning rate 5×10^{-6} , alpha 800, temperature 0.7, 4 epochs), compared against the standard configuration ($r = 32$, attention-only).

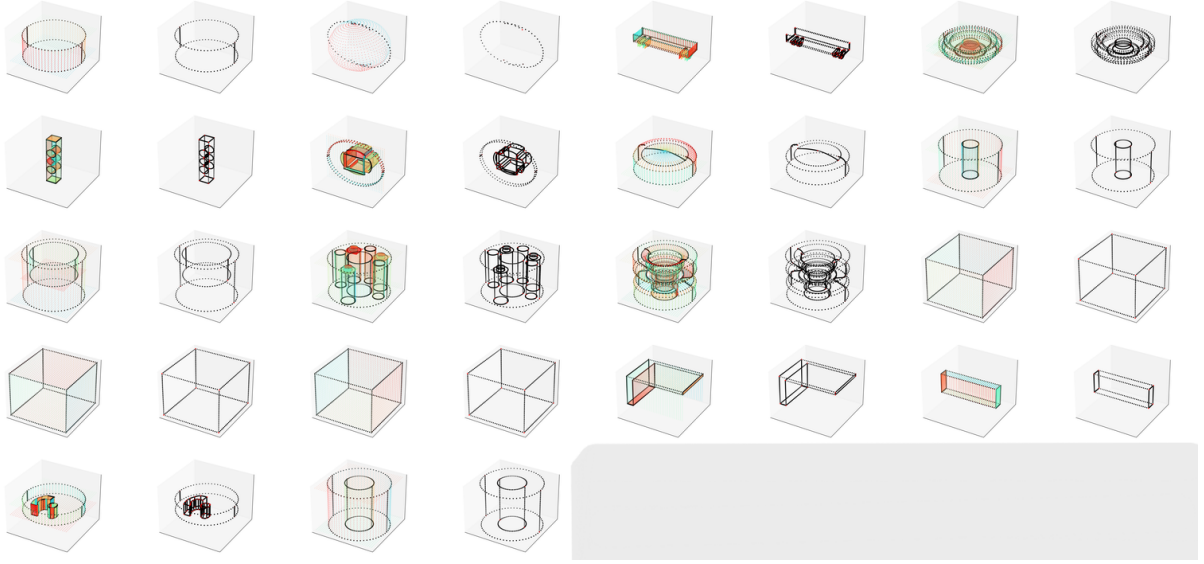


Figure 16: LoRA Rank and Adapter Scaling Comparison. Higher rank ($r = 200$, alpha 800) with feed-forward adapters vs. baseline attention-only ($r = 32$). LR: 5×10^{-6} , epochs: 4, T: 0.7.

5 Discussion of Research Questions

5.1 RQ1: Capabilities and Limitations of AutoBrep

The base AutoBrep checkpoint demonstrates impressive capabilities for unconditional B-Rep generation. As shown in Fig. 8, the model generates coherent, geometrically complex solids with 50–100+ faces and diverse topological structures. These structures are typically watertight, substantially outperforming prior diffusion-based approaches like BrepGen.

5.2 RQ2: Efficient Domain Adaptation with Limited Data

LoRA successfully enables data-efficient fine-tuning to specialized domains. With only 600 samples of cylindrical or cuboid solids, fine-tuned models achieved $\approx 90\%$ decodability into valid STEP files, requiring minimal computational overhead (single GPU, ≈ 0.5 hours).

However, LoRA functions as a *distribution sharpener* rather than a distribution shifter: it concentrates probability mass onto geometric modes already present in the base model rather than learning genuinely novel features. This means the base model must already contain (in distributed form) the characteristics of the fine-tuning domain.

5.3 RQ3: Sequential and Sparse (Auto)completion

Despite using causal masking architecturally, the base model fails at sequence completion tasks. When provided with partial B-Rep sequences (e.g., 20% of token sequence of training data solid), next-token predictions generate degenerate geometry that does not reconstruct valid solids (Fig. 15). This failure stems from the unconditional training regime: without explicit masked prediction or conditional objectives during pre-training, the model has not learned to reason about interdependencies between partial geometric constraints.

5.4 Limitations and Future Research

Three key constraints emerged from this investigation:

- **Conditional / Masked Training:** Retraining with geometry tokens enabled and masked prediction objectives should enable autocompletion and sequence continuation.
- **Larger Base Models:** Larger models trained on more data would contain richer feature spaces, potentially expanding what LoRA adapters can amplify.
- **Combined Transformer-Diffusion Architecture:** Combining autoregressive generation (long-range dependencies) with diffusion-based refinement (probability distribution shaping) might overcome mode-concentration limitations, as demonstrated in state-of-the-art image generation.

6 Conclusion

This thesis investigated parameter-efficient domain adaptation of the AutoBrep generative CAD model Xu et al. (2025) using Low-Rank Adaptation (LoRA) Hu et al. (2021), focusing on three research questions. Through systematic experimentation we find that:

1. **AutoBrep’s base capabilities** far exceed prior diffusion-based methods (Chapter 4), producing watertight, topologically complex solids with up to 100+ faces in a single autoregressive pass.
2. **LoRA enables rapid, data-light adaptation** to specialized domains (cylinders, cuboids with holes) using only $\approx 1\%$ additional parameters and 600 training samples, achieving $\approx 90\%$ decodable-solid rate. However, LoRA acts strictly as a *distribution sharpener* (Section 4.3): it can amplify geometric motifs already present in the base model’s latent feature space but cannot introduce genuinely novel primitives. Adapter placement and rank beyond $r = 16$ yield negligible gains, confirming that the bottleneck lies in the base model’s pre-trained geometry coverage, not in adapter capacity.
3. **Sequence completion tasks fail** across all fine-tuning regimes (Section 4.5). Despite the causal transformer architecture, the unconditional pre-training objective left the model unable to infer missing geometry from partial token sequences—an effect that LoRA cannot remedy without explicit conditional training objectives.

In essence, LoRA fine-tuning of AutoBrep solves the practical problem of *style-consistent generation* within a known shape language when the domain’s geometric vocabulary already exists in the base model. It does **not** solve the problem of learning topologies absent from the original training data, nor does it enable (auto)completion without explicit training modifications (see Chapter 5).

References

- Fan, Jiajie et al. (2024). “NeuroNURBS: Learning Efficient Surface Representations for 3D Solids”. In: *arXiv preprint*. arXiv: [2411.10848 \[cs.CV\]](#).
- Hu, Edward X. et al. (2021). “LoRA: Low-Rank Adaptation of Large Language Models”. In: *arXiv preprint*. arXiv: [2106.09685 \[cs.CL\]](#).
- Liu, Yilin et al. (2025). “HoLa: B-Rep Generation using a Holistic Latent Representation”. In: *ACM Transactions on Graphics (TOG)* 44.4. DOI: [10.1145/3730842](#). arXiv: [2504.14257 \[cs.GR\]](#).
- UnSloth (2024). *LoRA Hyperparameters Guide*. <https://www.unsloth.ai/docs/get-started/fine-tuning-llms-guide/lora-hyperparameters-guide>. Accessed: April 2026.
- Xu, Xiang et al. (2024). “BrepGen: A B-rep Generative Diffusion Model with Structured Latent Geometry”. In: *ACM SIGGRAPH 2024 Conference Papers*. Vol. 43. 4, 119:1–119:18. DOI: [10.1145/3658129](#). arXiv: [2401.15563 \[cs.CV\]](#).
- Xu, Xiang et al. (2025). “AutoBrep: Autoregressive B-Rep Generation with Unified Topology and Geometry”. In: *SIGGRAPH Asia 2025 Conference Papers*. DOI: [10.1145/3757377.3763814](#). arXiv: [2512.03018 \[cs.CV\]](#).

A Appendix

A.1 AI Use Declaration

To ensure transparency and uphold academic integrity, the following table outlines the artificial intelligence (AI) tools utilized during the preparation of this report. This declaration aligns with ETH Zurich’s guidelines on the responsible use of AI in scientific writing.

Table 4: AI Use Declaration

AI-Based Tool		Use Case	Scope	Remarks
GitHub Copilot	Claude Haiku 4.5	Code and table generation	boilerplate code and latex formatting	-

Declaration of Origin



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

- ☒ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies¹.
- ☒ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Generative Modeling of Structured CAD Geometry Using Transformer and VAE Frameworks

Authored by:

If the work was compiled in a group, the names of all authors are required.

Last name(s):

Lochmann

First name(s):

Sebastian

With my signature I confirm the following:

- I have adhered to the rules set out in the [Citation Guidelines](#).
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date

24.4.26, Zurich

Signature(s)

S. Lochmann

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

¹ For further information please consult the ETH Zurich websites, e.g. <https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html> and <https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html> (subject to change).